Torus 2D hydro simulation, pseudo-Newtonian

Two-dimensional (axially symmetric) hydrodynamical simulation of a RIAF without cooling. The calculations begin from an equilibrium configuration consisting of a thick torus (Papaloizou-Pringle) with constant specific angular momentum. Accretion is induced by the addition of a small anomalous azimuthal shear stress which.

Code units

$$G = M = 1$$

$$c = \sqrt{2}$$

such that the Schwarzschild radius is $r_S = \frac{2GM}{c^2} = 1$.

We usually parametrize time in units of the orbital time at the radius of maximum density (or pressure),

$$t_{\rm orb}(r_0) = \frac{2\pi r_0}{v_\phi(r0)}$$

where $v_\phi(r0)$ is the Keplerian velocity for the appropriate potential at r_0.

Torus parameters

The torus extends from $r_{\rm min} = 60$ to $r_{\rm max} \approx 300$. To achieve the desired torus, we set $r_0 = 100$.

The maximum density is $\rho_{\rm max} = 1$ and the minimum density in the atmosphere is 10^{-4}.

The adiabatic index is $\gamma = 5/3$.

Physics

Viscous stress tensor

Parametrized following Stone et al. (1999) with the kinematic viscosity given by

$$\nu = \alpha \frac{c_s^2}{\Omega_k} \approx \alpha r^{-1/2}$$

as appropriate for a RIAF. The shear stress is defined in file `visc_nu.c`.

Gravity

GR effects are incorporated via the Paczynski-Wiita potential,

$$\Phi = -\frac{GM}{r - r_S}$$

Notice that the Keplerian velocity is not simply $\Omega_k = \left(\frac{GM}{r^3}\right)^{1/2}$ with this pseudo-Newtonian potential but is instead given by

$$\Omega_k = \left(\frac{GM}{r}\right)^{1/2} \left(\frac{1}{r - r_S}\right)$$

The potential is defined in `init.c`.

Radiative cooling

The cooling is implemented in `radiat.c` where it is taken into account three cooling processes following Narayan & Yi 1995: bremmstrahlung (Q_{brem}), synchrotron (Q_{syn}) and synchrotron self-compton (Q_{ssc}). These processes are functions of the distance from the black hole (R), the electronic number density of the gas (n_e) and the electronic temperature (T_e). The cooling is implemented as following: first we generate a cooling table containing the values of the cooling rates of any combination of R, n_e and T_e, then we feed the `radiat.c` file by taking the values of R, n_e and T_e from the simulation and searching the table the corresponding cooling rate value.

For this example simulation we used $\frac{P_{gas}}{P_{mag}} = \beta = 10$, which is used to mimic a local and randomly oriented magnetic field since this is a HD simulation. The β parameter is implemented in `init.c` using the relation that $P_{tot} = P_{gas} + P_{mag} = \frac{1+\beta}{\beta} P_{gas}$.

Electronic Temperature

The hot accretion flow is two-temperature, with electron temperature being much lower than ion in the innermost regions of the accretion flow. This is due to the fact that electron cool much faster than ion and the different adiabatic index between electrons and ions in that region.

As pluto does not solve both fluids, we have to use an approximation for the calculation of the electron temperature. The pressure of the gas can be expressed as

$$P_{gas} = \frac{\rho k_b}{m_u} \left(\frac{T_i}{\mu_i} + \frac{T_e}{\mu_e}\right)$$

but pluto gives the total pressure $P_{tot} = P_{gas} + P_{mag} = \frac{\beta+1}{\beta} P_{gas}$ and we use an expression that relates T_i with T_e given by Wu et al. 2016

$$T_e = \frac{T_i}{(r/r_o)^{-k} + 2}$$

wherer k is a tunning parameter that here is equal 1.

Then, substituting P_{tot} in the equation of P_{gas} and using the relation of the temperatures we get a good approximation for the electron temperature.

Coordinate system and grid

Coordinate system: spherical polar

Resolution: $N_r \times N_\theta = 400 \times 400$ (cf. `pluto.ini`)

Radial range: $r = 1.3 - 400$ spanning three orders of magnitude in radius

Non-uniform grid in θ

A non-uniform grid in the θ-direction is adopted, because we are not necessarily interested in having resolution towards the poles, since we are not simulating relativistic jets. According to Moscibrodzka et al. (2014), the jet is produced inside a cone of angle 25∘. Therefore, compared to a uniform grid in θ, our approach is to have 1/3 of the uniform grid resolution within the "jet cone".

The formula for calculating the required number of elements inside one half of the jet cone is the following:

$$N_\theta(\text{jet}) = N_\theta(\text{total}) \left(\frac{25^\circ}{180^\circ} \right) (\text{reduction factor})$$

where we typically use `reduction factor = 1/3`.

For a concrete example, suppose we choose $N_\theta = 100$ and decide to decrease the resolution to 1/3 within 25∘ of the poles. According to the formula above, one has $n = 5$ inside each half of the cone. Therefore, we would have 5 elements in the upper half of the cone, 5 in the second half and 90 elements in the rest of the theta domain.

In Pluto, you can insert this modified resolution in `pluto.ini` as follows:

```
X2-grid  3    0.0  5 u 0.436 90 u 2.705 5 u 3.1415
```